

ANÁLISIS DEL SISTEMA

1. ¿Qué propiedades y métodos tendrá cada clase?

- **Clase Ubicacion:**
 - **Propiedades:** codigo, nombre, direccion, nivelRiesgo, estado.
 - **Métodos:** Constructor de la clase, Getters para todas las propiedades, Setters para nivelRiesgo y estado, método auxiliar de validación y toString().
- **Clase Pista:**
 - **Propiedades:** codigo, descripcion, tipoEvidencia, nivelImportancia, nivelConfiabilidad.
 - **Métodos:** Constructor de la clase, Getters para todas las propiedades, Setters para descripcion, tipoEvidencia y setNiveles(), método auxiliar validarNiveles() y toString().
- **Clase Caso:**
 - **Propiedades:** nombreCaso, codigoidentificacion, detectiveResponsable, ubicaciones y pistas.
 - **Métodos:** Constructor de la clase, Getters principales, registrarUbicacion(), consultarUbicacion(), modificarUbicacion(), descartarUbicacion(), registrarPista(), buscarPista(), modificarPista(), eliminarPista(), cantidadUbicacionesRegistradas(), cantidadEspaciosDisponibles(), ubicacionMayorRiesgo(), pistaMayorImportancia(), pistaMayorConfiabilidad() y promedioImportanciaPistas().
- **Clase Main (Driver Program):**
 - **Métodos:** main(String[] args) que controla el menú interactivo, la entrada/salida de consola mediante Scanner y la captura global de excepciones.

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Clase Ubicacion

- codigo: String
- nombre: String
- direccion: String
- nivelRiesgo: int
- estado: String
- Getters / Setters: Devuelven el tipo respectivo de cada atributo o void.
- toString(): String

Clase Pista

- codigo: String
- descripcion: String
- tipoEvidencia: String
- nivelImportancia: int
- nivelConfiabilidad: int
- Getters / Setters: Devuelven el tipo respectivo de cada atributo o void.
- validarNiveles(int, int): void (método privado)
- toString(): String

Clase Caso

- nombreCaso: String
- codigoidentificacion: String
- detectiveResponsable: String
- ubicaciones: Ubicacion[] (Arreglo unidimensional)
- pistas: ArrayList<Pista>
- registrarUbicacion(int, Ubicacion): void
- consultarUbicacion(int): Ubicacion
- modificarUbicacion(int, int, String): void
- descartarUbicacion(int): void
- registrarPista(Pista): void
- buscarPista(String): Pista
- modificarPista(String, String, String, int, int): void
- eliminarPista(String): void
- cantidadUbicacionesRegistradas(): int
- cantidadEspaciosDisponibles(): int
- ubicacionMayorRiesgo(): Ubicacion
- pistaMayorImportancia(): Pista
- pistaMayorConfiabilidad(): Pista
- promedioImportanciaPistas(): double

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?

La propiedad ubicaciones dentro de la clase Caso debe implementarse utilizando un **arreglo básico estático** (Ubicacion[]).

- **Tipo de objetos que almacenará:** Instancias de la clase Ubicacion.
- **Tamaño fijado:** Exactamente **5 elementos** (new Ubicacion[5]), representando las ubicaciones finitas asignadas a la investigación al inicio del caso.

4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?

La propiedad pistas dentro de la clase Caso se implementa obligatoriamente con un **ArrayList<Pista>**.

- **Tipo de objetos que almacenará:** Objetos dinámicos del tipo Pista.
- **Razón:** Permite almacenar una cantidad indeterminada y cambiante de evidencias recolectadas en el transcurso de las investigaciones, facilitando inserciones, búsquedas y eliminaciones sin un límite predefinido de memoria.

5. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?

Alicando los principios fundamentales del encapsulamiento:

- **Atributos / Propiedades:** Todos deben ser definidos con visibilidad privada (private) para proteger el estado interno de los objetos y evitar manipulaciones no autorizadas desde el exterior.
- **Métodos públicos (public):** Constructores, Getters, Setters, métodos de operaciones de negocio (registro, búsqueda, reportes, modificaciones) y toString().
- **Métodos privados (private):** Métodos de apoyo interno como validarNiveles() dentro de la clase Pista.

6. ¿Qué parámetros serán requeridos por los métodos en sus clases?

- registrarUbicacion(int posicion, Ubicacion ub): La posición destino (0 a 4) y el objeto de la nueva ubicación.
- consultarUbicacion(int posicion): El índice a consultar dentro del arreglo.
- modificarUbicacion(int posicion, int nuevoRiesgo, String nuevoEstado): La posición a alterar y los nuevos valores a actualizar.
- descartarUbicacion(int posicion): El índice que se desea vaciar (null).
- registrarPista(Pista pista): El objeto Pista validado para almacenar.
- buscarPista(String codigo): El código identificador de la evidencia.
- modificarPista(String codigo, String nuevaDesc, String nuevoTipo, int nuevaImp, int nuevaConf): El código identificador y los nuevos valores.
- eliminarPista(String codigo): El código identificador de la pista a remover.

7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?

- **Valores Iniciales:** Se asignan a través de los **constructores** de cada clase al instante de instanciarlos. En la clase Caso, el arreglo ubicaciones se inicializa con 5 posiciones con valor por defecto null y el ArrayList de pistas nace vacío (size() == 0).
- **Validaciones Obligatorias de Estado:**
 1. En Ubicacion: nivelRiesgo debe estar obligatoriamente en el rango comprendido de 1 a 10.
 2. En Pista: nivelImportancia debe estar en el rango [1, 10] y nivelConfiabilidad en el rango [0, 100].
 3. En Caso: Antes de registrar una pista se valida que no exista otra con el mismo código.

8. ¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?

Se utiliza la comparación explícita de referencias contra null mediante la sintaxis:

Java

```
if (ubicaciones[posicion] != null) { ... }
```

Al recorrer el arreglo (por ejemplo en reportes o búsquedas de riesgos), un ciclo for evalúa cada casilla i. Si la posición contiene null, el sistema omite la invocación de métodos para prevenir un fallo de tipo NullPointerException.

9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

1. **Búsqueda:** Se realiza un recorrido iterativo (for-each) comparando los códigos mediante `codigo.equalsIgnoreCase(codBuscar)`.
2. **Modificación:** Se localiza la referencia del objeto mediante el método de búsqueda; una vez obtenido el objeto, se actualizan sus atributos invocando los métodos setters (`setDescripcion()`, `setNiveles()`), activando las validaciones correspondientes.
3. **Eliminación:** Se busca el objeto Pista asociado al código introducido. Si se encuentra, se utiliza el método propio del framework de colecciones `pistas.remove(pista)`.

10. ¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

- **Situaciones de Error e Identificación de Excepciones:**

- **InputMismatchException:** Ocurre en el Main cuando el usuario ingresa una cadena de texto o un carácter no numérico al solicitarle una opción del menú o valores enteros.
- **IllegalArgumentException:** Ocurre en las clases Ubicacion y Pista al enviar un nivel de riesgo (< 1 o > 10), importancia (< 1 o > 10) o confiabilidad (< 0 o > 100) fuera de rango, o cuando se intenta registrar una pista con un código duplicado.
- **IndexOutOfBoundsException:** Ocurre al intentar acceder a una posición menor a 0 o mayor o igual a 5 dentro del arreglo estático de ubicaciones.
- **IllegalStateException:** Ocurre cuando el usuario intenta registrar una ubicación en una posición del arreglo que ya se encuentra ocupada (distinta de null).
- **Uso de Bloques try-catch y finally:**
 - **En el Main:** Cada case interactivo del menú cuenta con bloques try-catch específicos para aislar fallos, capturar las excepciones arrojadas por la lógica de negocios y mostrar un mensaje descriptivo de error sin que el programa colapse.
 - **Uso del Bloque finally:** En la opción 2 (Registrar ubicación), se implementa un bloque finally que imprime la confirmación "(Fin del intento de registro de ubicación)". Este bloque se ejecuta obligatoriamente de forma garantizada tanto si el registro fue exitoso como si falló por un valor erróneo o una posición ocupada.

